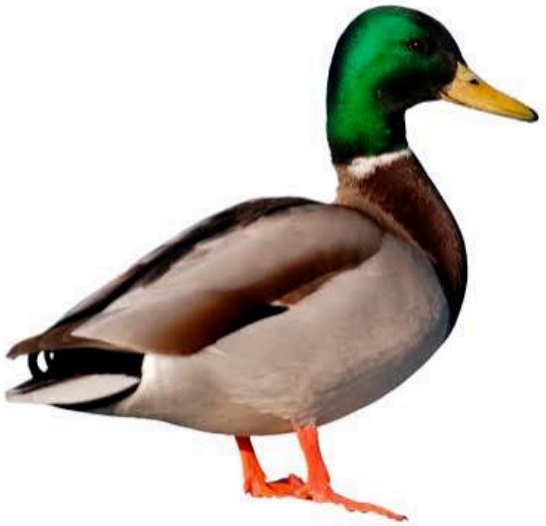
Ciência de Dados Aplicada ao Direito I

Aula 12

Variáveis, condições e respostas em Python



Prof. Bruno da Cunha de Oliveira - 2026/2

Qual é o erro, e qual é o conserto?

Em cada célula, localizem o erro e proponham o conserto antes de eu rodar.

```
# 5a
print("Petição inicial')
# 5b
prazo = 15 * 2 prazo em dobro para a Defensoria
# 5c
comarca = "São Paulo"
print(Comarca)
```

Qual é o erro, e qual é o conserto?

RESPOSTA

Célula	Última linha	Conserto
5a	`SyntaxError`	fechar com a mesma aspa que abriu
5b	`SyntaxError`	`#` antes de `prazo em dobro`
5c	`NameError`	`comarca`, com minúscula, como foi criada

De baixo para cima, sempre: a última linha diz o tipo, a seta diz onde. O conserto precisa fazer a célula rodar e produzir o resultado pretendido.

Vocês já falam Python

Insper

O pseudocódigo, em Python

Pseudocódigo	Python
`idade_autor = 67`	`idade_autor = 67`
`resultado = "PROCEDENTE"`	`resultado = "PROCEDENTE"`
`VERDADEIRO` / `FALSO`	`True` / `False`
`NULO`	`None`
`partes = ["Ana", "Bruno"]`	`partes = ["Ana", "Bruno"]`
`{"numero": "0000000-00.2026"}`	`{"numero": "0000000-00.2026"}`
`idade_autor >= 60 E tutela_deferida`	`idade_autor >= 60 and tutela_deferida`

Variável, valor, tipo, lista, dicionário, conectivo: tudo isso vocês já escrevem. O que muda é uma palavra ou outra.

Nome de variável: as regras

Qual nome impede o programa de rodar? Como corrigir sem mudar os dados?

```
nome completo = "Ana Silva"
print(nome_completo)
```

Nome de variável: as regras

RESPOSTA

```
nome_completo = "Ana Silva"
print(nome_completo)
```

```
Ana Silva
```

Sem espaços ou hífen no nome, sem começar por número e sem usar palavras reservadas. Vamos usar minúsculas, sem acentos, com _ entre palavras. Isso é convenção: `réu` é válido, e `nome_completo` e `Nome_completo` são nomes diferentes.

Juntar os nomes sem colar as palavras

O cadastro tem dois campos. Este código produz o nome completo como deve aparecer na mensagem?

```
nome = "Ana"
sobrenome = "Silva"
print(nome + sobrenome)
```

Juntar os nomes sem colar as palavras

RESPOSTA

```
nome = "Ana"
sobrenome = "Silva"
print(nome + " " + sobrenome)
```

```
Ana Silva
```

O espaço também é texto. Entre números, `+` soma; aqui ele junta as duas partes com o espaço pedido.

No PrairieLearn: Desafio_I

Insper

Conferir o documento

Insper

O documento chegou?

O programa deve confirmar só documentos que chegaram. O que este código mostra? Ele respeita essa regra?

```
tem_documento = False
print("Documento recebido")
```

O documento chegou?

RESPOSTA

```
tem_documento = False
print("Documento recebido")
```

```
Documento recebido
```

O programa confirma o recebimento mesmo quando o dado diz que não chegou. A mensagem precisa depender de `tem_documento`.

if: confirmar só quando chegou

```
tem_documento = False
if tem_documento:
    print("Documento recebido")
```

`SE` vira `if`; os dois pontos abrem o bloco, e o recuo mostra o que depende da condição. Com `False`, esta célula não escreve nada.

Troquem para `True`: agora aparece `Documento recebido`. E como avisar quando o documento não chegou?

else: informar também a ausência

```
tem_documento = False
if tem_documento:
    estado = "recebido"
else:
    estado = "falta documento"
print(estado)
```

```
falta documento
```

O `else` responde ao caso em que a condição deu `False`. Agora cada caminho define `estado`, e o `print` mostra a resposta escolhida.

or: receber por qualquer um dos canais

Receber por e-mail ou no balcão basta. Com estes dados, o documento chegou?

```
por_email = False
no_balcao = True
tem_documento = por_email or no_balcao
print(tem_documento)
```

or: receber por qualquer um dos canais

RESPOSTA

```
por_email = False
no_balcao = True
tem_documento = por_email or no_balcao
print(tem_documento)
```

```
True
```

`or` é o `OU`: basta um canal. O valor guardado em `tem_documento` é a resposta que o `if` já usa. Se os dois forem `False`, o documento não chegou.

Uma decisão dentro da outra

Recebido ainda não quer dizer pronto: o documento também precisa estar assinado. A assinatura só é conferida se houver documento. O que aparece na tela?

```
tem_documento = True
assinado = False
if tem_documento:
    if assinado:
        estado = "pronto"
    else:
        estado = "falta assinatura"
else:
    estado = "falta documento"
print(estado)
```

Uma decisão dentro da outra

RESPOSTA

```
tem_documento = True
assinado = False
if tem_documento:
    if assinado:
        estado = "pronto"
    else:
        estado = "falta assinatura"
else:
    estado = "falta documento"
print(estado)
```

`tem_documento` é `True`: entra no bloco do primeiro `if`.

`assinado` é `False`: segue pelo `else` interno e guarda `"falta assinatura"` em `estado`.

O `else` externo não roda: o documento existe. O `print`, fora dos dois blocos, mostra:

```
falta assinatura
```

Indentação: qual mensagem depende do if?

A conferência precisa registrar que terminou, mesmo quando falta documento. Com `tem_documento = False`, qual versão faz isso?

Tudo dentro do if

```
if tem_documento:
    print("Documento recebido")
    print("Conferência encerrada")
```

A última mensagem fora do if

```
if tem_documento:
    print("Documento recebido")
print("Conferência encerrada")
```

À esquerda, nenhuma mensagem aparece. À direita, a última linha roda independentemente do recebimento. O recuo muda o que fica condicionado.

IndentationError: faltou o recuo

```
tem_documento = True
if tem_documento:
estado = "recebido"
```

```
IndentationError: expected an indented block after 'if' statement on line 2
```

O Python esperava o bloco depois dos dois pontos. Recuar `estado = "recebido"` para dentro do `if` corrige a sintaxe; depois é preciso conferir se essa é a regra pretendida.

O def já escrito

No notebook, a função já vem escrita para o corretor testar a regra com outras entradas. O seu código entra no espaço recuado:

```
def analisar_documento(tem_documento, assinado):
    # escreva a regra aqui

    return estado
```

Neste exemplo, a regra precisa definir `estado` em todos os caminhos. No exercício, preservem a assinatura, o `return` e os nomes pedidos. Executar a célula define a função; não significa que a resposta foi corrigida.

No PrairieLearn: exercicio2

Conferir os dados e responder

and e not: recebido, assinado e não cancelado

A triagem considera pronto o documento recebido e assinado. Mas um documento cancelado não pode seguir. Esta condição já trata o cancelamento?

```
tem_documento = True
assinado = True
cancelado = True
pronto = tem_documento and assinado
print(pronto)
```

and e not: recebido, assinado e não cancelado

RESPOSTA

```
tem_documento = True
assinado = True
cancelado = True
pronto = tem_documento and assinado and not cancelado
print(pronto)
```

```
False
```

`and` exige os requisitos juntos. `not cancelado` é `False`, então `pronto` é `False`. Se só `cancelado` mudar para `False`, `pronto` será `True`.

Parênteses: a assinatura vale para os dois canais

O documento pode chegar por qualquer canal, mas precisa estar assinado. Este código aceita um documento sem assinatura?

```
por_email = True
no_balcao = False
assinado = False
pronto = por_email or no_balcao and assinado
print(pronto)
```

Parênteses: a assinatura vale para os dois canais

RESPOSTA

```
por_email = True
no_balcao = False
assinado = False
pronto = (por_email or no_balcao) and assinado
print(pronto)
```

```
False
```

Os parênteses reúnem os canais. A assinatura passa a ser exigida para qualquer um deles. `or` sem esse agrupamento permitia que o e-mail dispensasse a assinatura.

==: a situação chegou como texto

```
situacao = "ASSINADO"
assinado = situacao == "ASSINADO"
print(assinado)
```

```
True
```

O formulário entrega um texto, mas a regra precisa de um booleano. `==` faz a comparação; `=` guarda sua resposta em `assinado`. O restante da regra continua usando essa variável.

SyntaxError: atribuir não é comparar

```
situacao = "ASSINADO"
if situacao = "ASSINADO":
    print("Assinatura conferida")
```

```
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

Um = não faz a pergunta que precisamos. Trocar por == compara a situação recebida com o texto esperado.

O texto recebido é exatamente o esperado?

A situação parece dizer ASSINADO. Por que a comparação devolve outro resultado?

```
situacao = "ASSINADO "
print(situacao == "ASSINADO")
print("assinado" == "ASSINADO")
```

O texto recebido é exatamente o esperado?

RESPOSTA

```
situacao = "ASSINADO "
print(situacao == "ASSINADO")
print("assinado" == "ASSINADO")
```

```
False
False
```

Há um espaço no fim da primeira situação; na segunda comparação, a caixa é diferente. A comparação confere todos os caracteres. Corrigir um registro errado não é trocar `==` por `=`.